

Git Advanced

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-03-17

- Branches
- Timetravel
- Branch Operations
- Conflict Management
- Git workflows

Bookkeeping

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git advanced	24/3	Y	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: Python packaging ecosystem	21/4	N	RZ D8
10: Testing and debugging	28/4	Y	RZ D8
11: Joker lecture	5/5	Y	ML H41.1
12: OOP	12/5	N	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment

Important project dates

Keep these dates in mind for your project planning!

Milestone	Date	TODO
Project chosen	13/3	Done
Project Structured	27/3	Submit (markdown) file with project structure
Implementing	31/3	8 weeks to code and refine project
Project draft	15/5	Pre-submission of project for feedback
Project submission	29/5	Final submission of project on Git (tag)

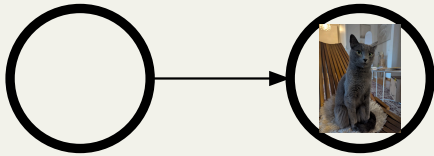
Branching off

Last lecture: Git enables version control by storing **snapshots** of your files that you can revert to if needed.

- **Git History** is the timeline of your project containing all **commits**
- A **commit** is a snapshot of your project at a point in time
- Each commit has a **unique hash** identifier
- Commits store: author info, date, message, and parent commit pointer(s)

Today: Understanding commit relationships and branching strategies

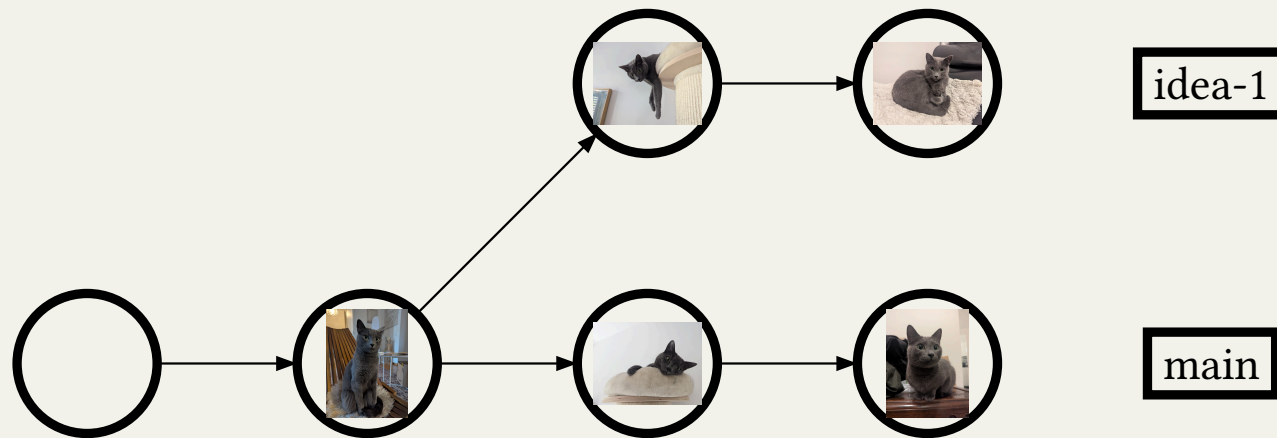
Imagine you have a repository that hosts a story about a cat. You want to extend the story, but you're not sure which direction to take.



Introduction into branches

Branching off

Imagine you have a repository that hosts a story about a cat. You want to extend the story, but you're not sure which direction to take.



You can create branches to explore different storylines! Both **branches** now include **seperate versions** of the story.

In your table/group discuss for 3 minutes:

- When should you make a branch?
- What are some examples of branches you could create in your project?
- What are some potential risks of branching?

Branches are one of Git's most powerful features! They:

- **Provide Isolation** $\Rightarrow$ **Parallel Development** in teams (less conflicts)
- Allows for safe **unstable experimentation**
- Ensure you keep a **stable**, working version of your project!

Branches are one of Git's most powerful features! They:

- **Provide Isolation** $\Rightarrow$ **Parallel Development** in teams (less conflicts)
- Allows for safe **unstable experimentation**
- Ensure you keep a **stable**, working version of your project!

Potential risks of branching:

- **Merge conflicts** when multiple branches modify the same code
- **Long-lived branches** can diverge significantly, making integration harder
- **Overhead** of managing many branches simultaneously
- **Confusion** about which branch contains the most recent work

Git provides two commands for switching between branches:

git checkout (older, versatile)

- Can switch branches, restore files, create branches
- `git checkout branch_name` — switch to a branch
- `git checkout -b new_branch` — create and switch to new branch
- Can be confusing due to multiple purposes

git switch (newer, focused)

- Introduced in 2019 only for switching branches
- `git switch branch_name` — switch to a branch
- `git switch -c new_branch` — create and switch to new branch
- Clearer intent, recommended for branch operations

Follow these steps to practice creating and using branches:

1. Create a new folder and initialize a Git repository
2. Add a new text file called `story.txt` with some content (e.g., about a cat)
3. Stage and commit your work
4. Create a new branch (give it a name) and switch to it
5. Use `git branch` and `git log` to observe your branches and commit history

Tip: Use `git status` at any time to verify which branch you're currently on

The `git branch` command is essential for branch management in Git:

Key operations:

- **List branches:** `git branch` (shows all local branches, marks current with `*`)
- **Create a branch:** `git branch <branch-name>` (creates branch without switching)
- **Delete a branch:** `git branch -d <branch-name>` (safely deletes merged branches)

The **`branch`** command will help you clean up completed work by removing obsolete branches.

Practice working with branches and observe how Git preserves your work:

1. Switch to your newly created branch (from before)
2. Open `story.txt` in an editor and make a small change
3. Stage and commit your change and **close the editor**
4. Switch back to the main branch using `git checkout main`
5. Reopen `story.txt` — **Observe**: What happened to your changes?
6. Switch back to your feature branch and reopen `story.txt` again — **Observe**: What happened now?

Key insight: Your changes are still there! Git switches the file content based on which branch you're on.

Git is **non-destructive** by default—your work on each branch is preserved.

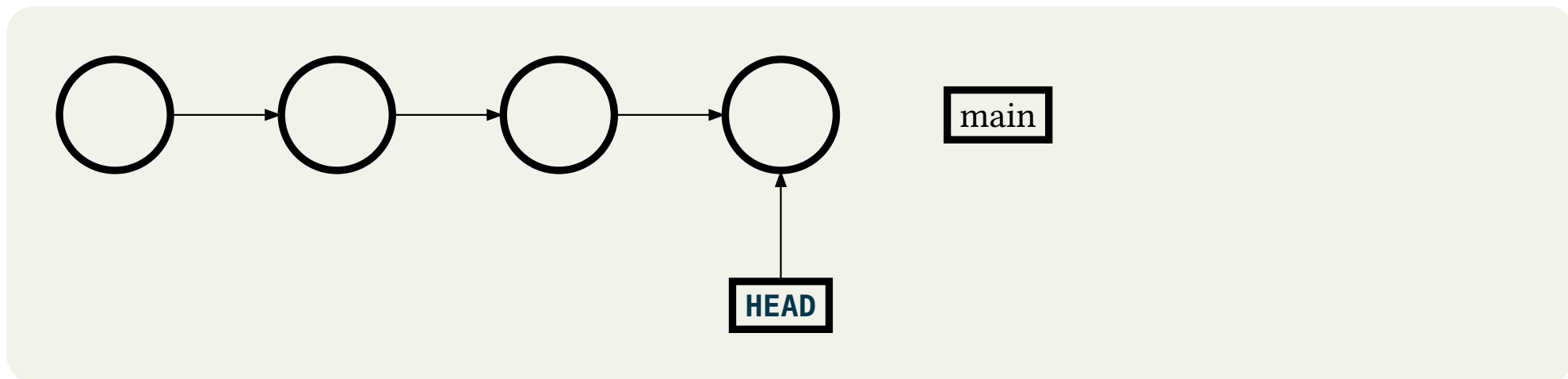
Key insight: Your changes are still there! Git switches the file content based on which branch you're on.

Git is **non-destructive** by default—your work on each branch is preserved.

This means you can safely experiment on branches without fear of losing your main branch's work. Each branch maintains its own snapshot of the project.

Timetravel

We call the **newest** commit of a branch the **HEAD**.



You can move between commits using the `git checkout` command. All you need is the **commit hash**.

Step 1: List your commit history using `git log` and find the commit you want to jump to.

```
commit 7e388b81ffad388825936b77bc1ffe874ce4437 (HEAD -> main)
Author: Ian Wasser <iwasser@ethz.ch>
Date:   Mon Mar 2 09:38:13 2026 +0100
```

```
Hello
```

Step 3: Use the hash to checkout that commit:

```
git checkout 7e388 # usually the first 4-5 digits are sufficient
```

When you checkout a specific commit, your files reflect that moment in the project. You're now in a **detached HEAD state** — not on any branch. **You cannot make changes in this state.** To continue:

Option 1: Return to a branch

```
git checkout main
```

Option 2: Branch off

```
git checkout -b new_branch
```

This creates a new branch starting from this commit, allowing you to continue working.

Practice timetravel and branching from a detached HEAD state:

1. Open your `story.txt` file and add a new sentence, then commit your changes. Repeat a few times (>3 commits)
2. Use `git log` and choose a past commit
3. Checkout that older commit using its hash
4. Open `story.txt` and observe its state
5. Create a new branch from this point

You now have an alternative timeline branching from the past!

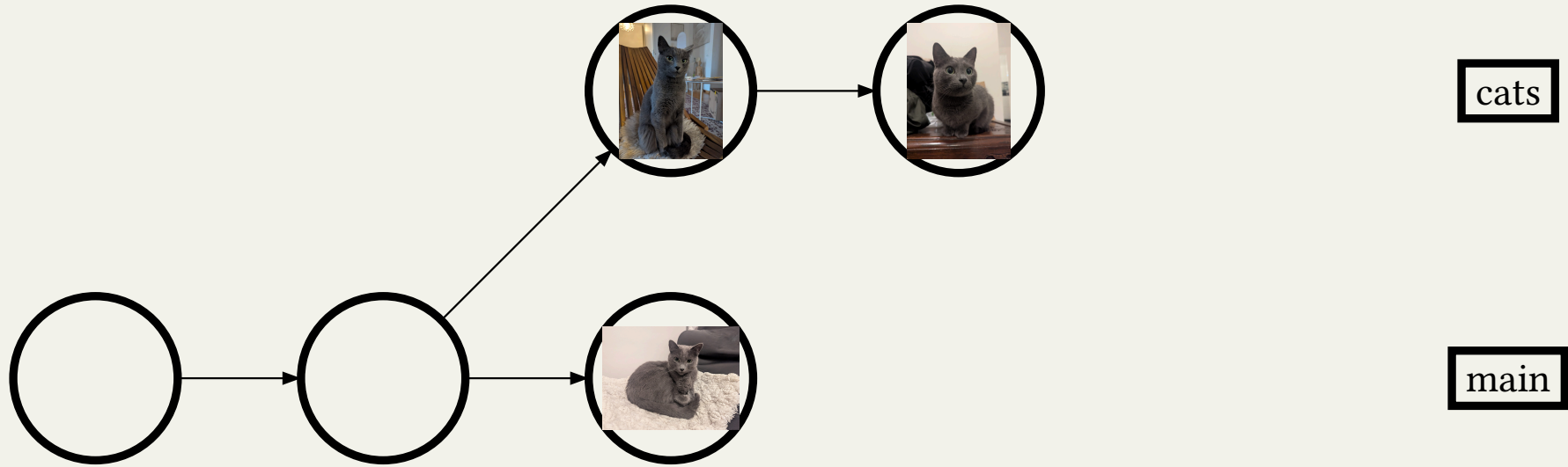
Branch Operations

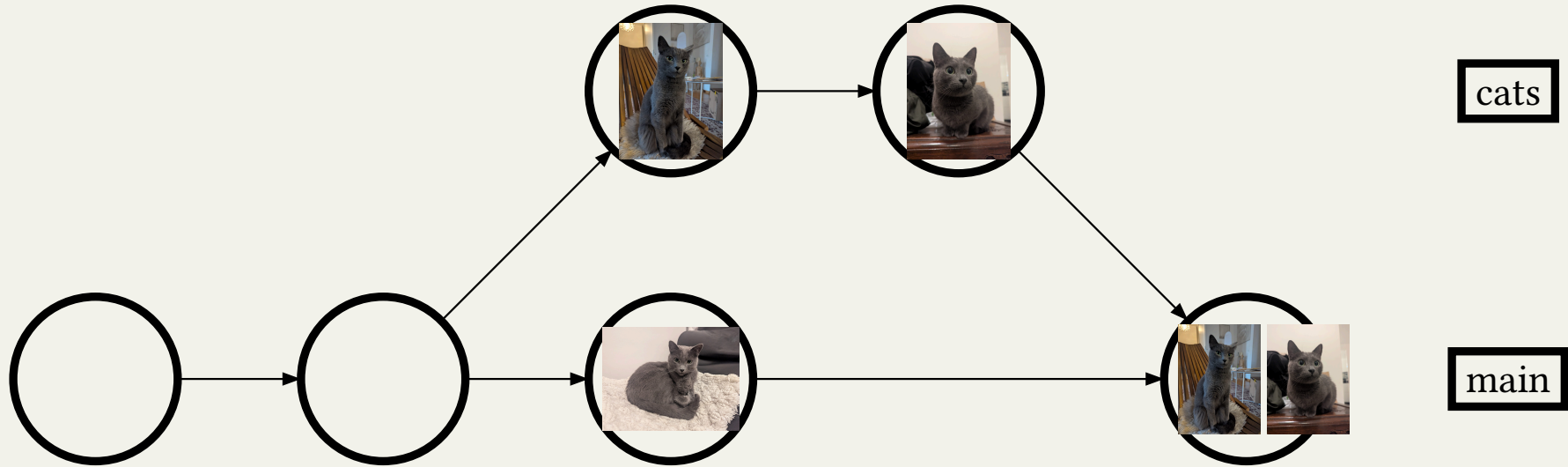
After you've branched off and worked on the project, you probably want to bring those changes back to your project.

We'll now explore two branch operations that allow us to do this:

- Merge
- Rebase

Merging





Now let's merge your `alternative_story` branch back into `main`:

1. Make sure you add some content to your `alternative_story` branch and commit it
2. Switch back to the `main` branch, open `story.txt` and add some new content
3. Stage and commit your changes on `main`
4. Attempt to merge your branch while in `main`: `git merge alternative_story`

What Does a Merge Conflict Look Like?

If you followed the steps properly, you should encounter a merge conflict! When you run the merge command and both branches modified the same file, Git will show something like:

```
git merge alternative_story  
  
Auto-merging story.txt  
CONFLICT (content): Merge conflict in story.txt  
Automatic merge failed; fix conflicts and then commit the result.
```

Conflicts

Conflicts may happen when editing the same files differently in the same branch.

File: story.txt

This is the story of a cat
called Gin!



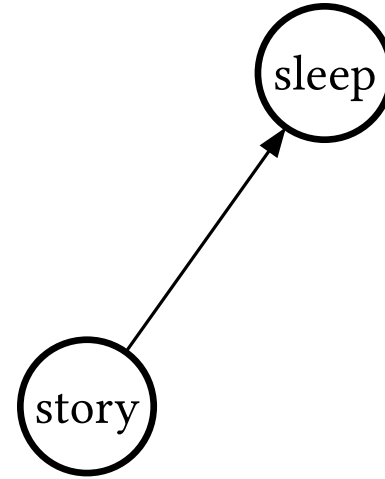
main

Conflicts

Merging

File: story.txt

This is the story of a cat
called Gin!
He loves to sleep.



cats

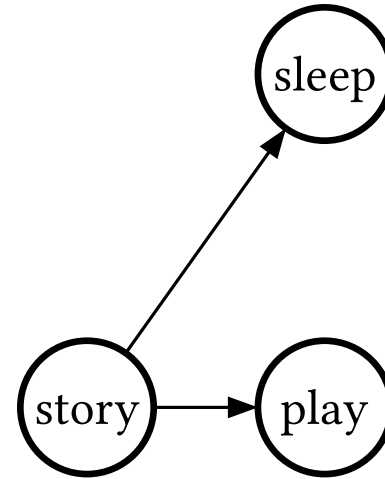
main

Conflicts

Merging

File: story.txt

This is the story of a cat
called Gin!
He plays a lot.



cats

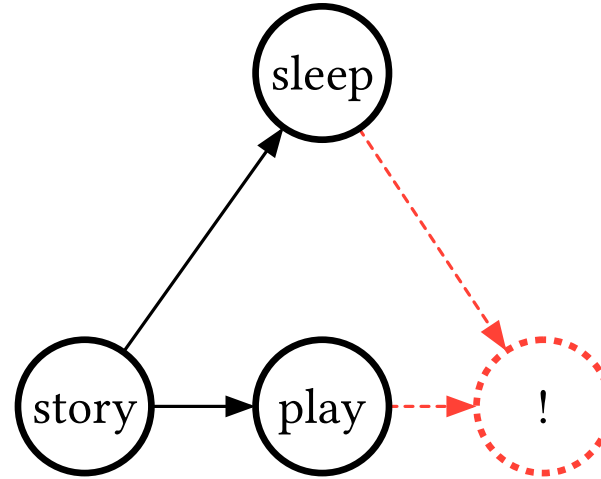
main

Conflicts

We try to merge

```
git merge cats
```

```
Auto-merging story.txt  
CONFLICT (content): Merge  
conflict in story.txt  
Automatic merge failed; fix  
conflicts and then commit the  
result.
```



cats

main

Conflicts

In `story.txt` you should find **conflict markers**

```
This is the story of a cat called Gin!  
<<<<<< HEAD  
He plays a lot.  
||||||| 8ffed99  
=====  
He loves to sleep.  
>>>>>> cats
```

Conflicts

In `story.txt` you should find **conflict markers**

```
This is the story of a cat called Gin!  
<<<<<< HEAD  
He plays a lot.  
|||||| 8ffed99  
=====  
He loves to sleep.  
>>>>>> cats
```

These are Git's way of showing you the conflicting changes from both branches. The content between `<<<<<< HEAD` and `=====` is from the current branch (main), while the content between `=====` and `>>>>>>` cats is from the branch being merged (cats).

Resolving conflicts

To resolve this issue we would simply rewrite the file to our liking. For example:

```
This is the story of a cat called Gin!  
<<<<<< HEAD  
He plays a lot.  
||||||| 8ffed99  
=====  
He loves to sleep.  
>>>>>> cats
```

```
This is the story of a cat called Gin!  
He plays and sleep a lot.
```

Resolving conflicts

You can then run

```
git add .  
git merge --continue
```

to resolve the issue and move on with the merge. After resolving conflicts, verify the merge was successful:

```
git status # Should show "nothing to commit, working tree clean"  
git log --oneline --graph --all # See the merge commit in history  
# or try:  
git status  
gitk --all
```


Resolving conflicts

You can then run

```
git add .  
git merge --continue
```

to resolve the issue and move on with the merge. After resolving conflicts, verify the merge was successful:

```
git status # Should show "nothing to commit, working tree clean"  
git log --oneline --graph --all # See the merge commit in history  
# or try:  
git status  
gitk --all
```


Resolving conflicts

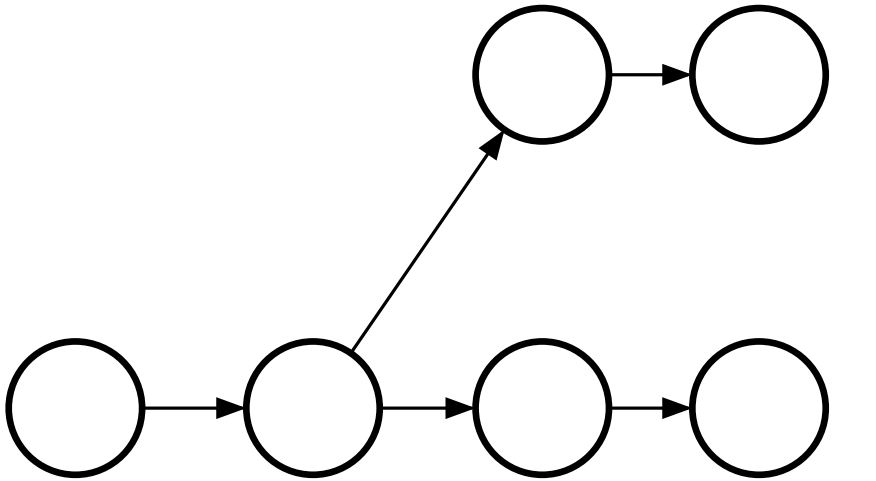
The merge creates a new commit with **two parent pointers** — one to each branch that was merged. This preserves the full history of both branches.

```
*    a1b2c3d (HEAD -> main) Merge branch 'alternative_story'
|\
| * d4e5f6g Added alternative ending
* | g7h8i9j Added main storyline
|/
* j0k1l2m Initial story
```

Rebasing

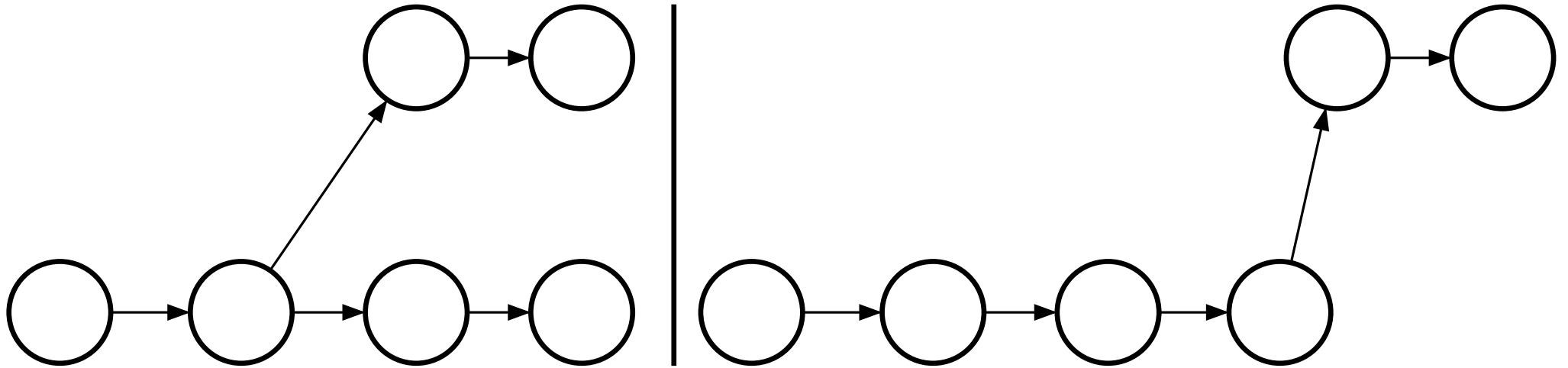
What is Rebase?

Rebase moves the **base** of a branch to another commit.



What is Rebase?

Rebase moves the **base** of a branch to another commit.

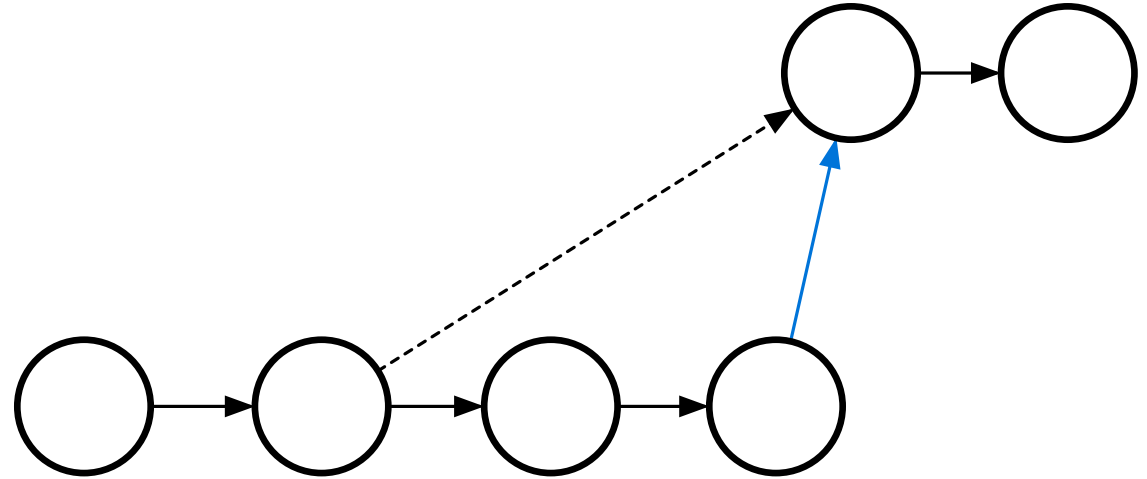


Rebasing Commands

You need to be on the branch you want to rebase.
You specify the commit you want to set the base to.

Here, we set base to be the latest commit on the main branch.

```
git rebase main
```



Rebase Conflicts

The **rebase command** might introduce some **merge Conflicts**. Can be handled same way as before; you need to resolve the **conflict markers** and run

```
git add . # stage the conflict files
git rebase --continue
```



Note:

These steps might need to be repeated if several conflicts exist.

Practice rebasing to update your branch with changes from main:

1. Switch to main and add another sentence to `story.txt`, then commit
2. Switch back to your `alternative_story` branch
3. Run `git log` on both branches to see they've diverged
4. While on `alternative_story`, run: `git rebase main`
5. Check `git log` again — your branch now includes main's latest changes!

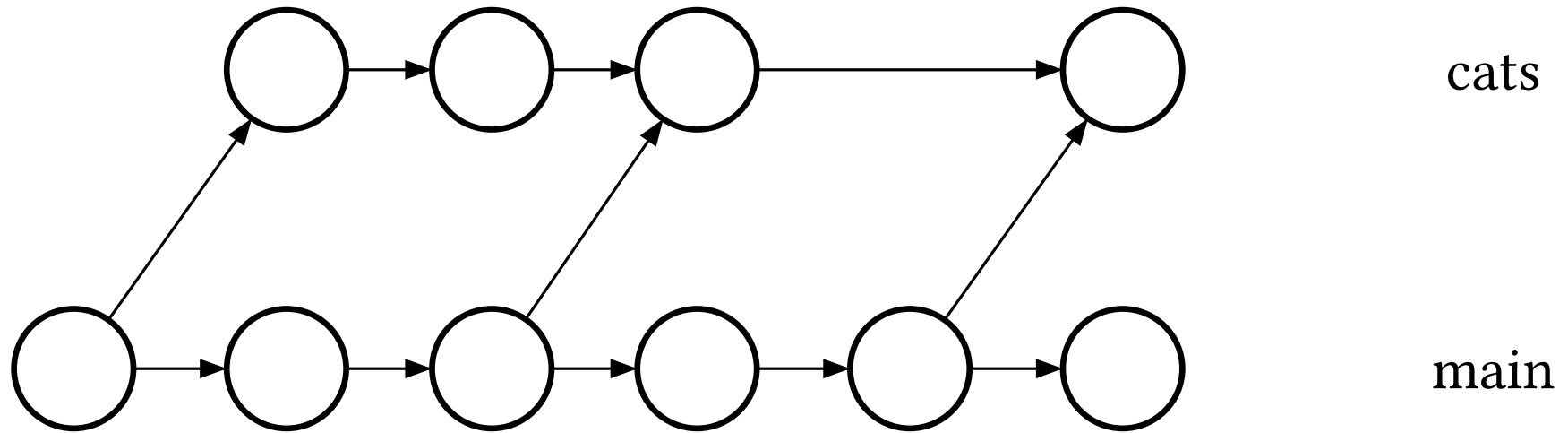
Tip: If you encounter conflicts during rebase, resolve them, then use `git add .` and `git rebase --continue`.

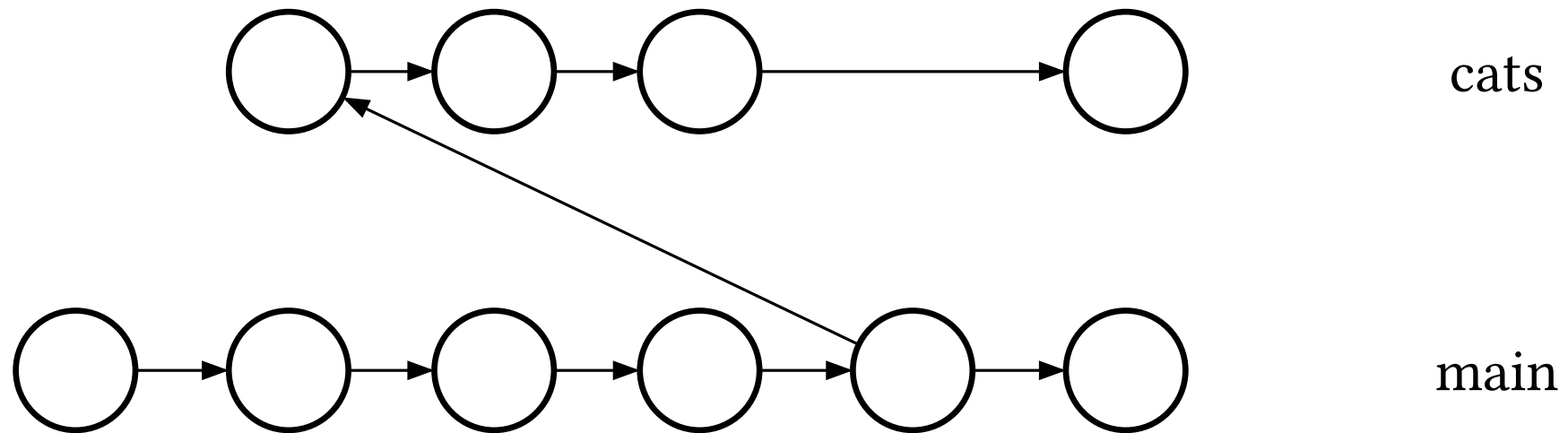
Difference between Merge and Rebase

They solve two different things:

- **Merge** merges one branch into another branch. This creates a new commit.
- **Rebase** updates the history of your branch (which may lead to confusion!).

Why not update with merge





Git workflows

What are Git Workflows?

A **Git workflow** is a strategy for how teams organize and manage their Git branches and commits.

Why do we need workflows?

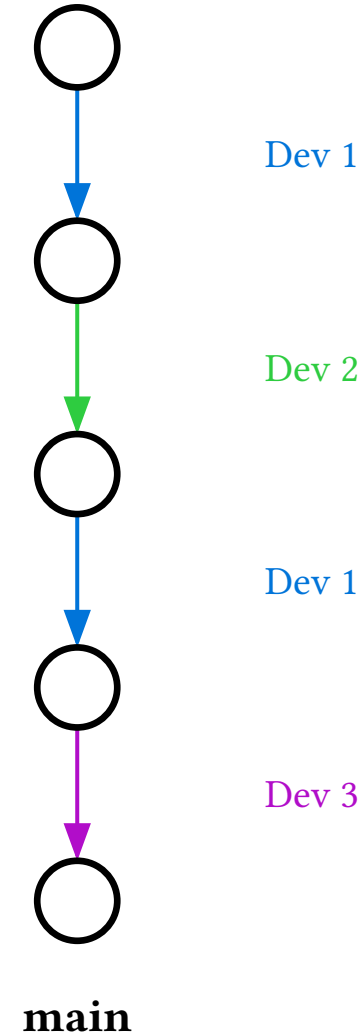
- **Consistency:** Everyone follows the same process
- **Reduce conflicts:** Clear rules prevent overlapping work
- **Quality control:** Built-in review and testing steps
- **Scale collaboration:** Easy to bring in new members

Key considerations:

- Team size and experience
- Project complexity
- Release frequency
- Risk tolerance
- Code review requirements

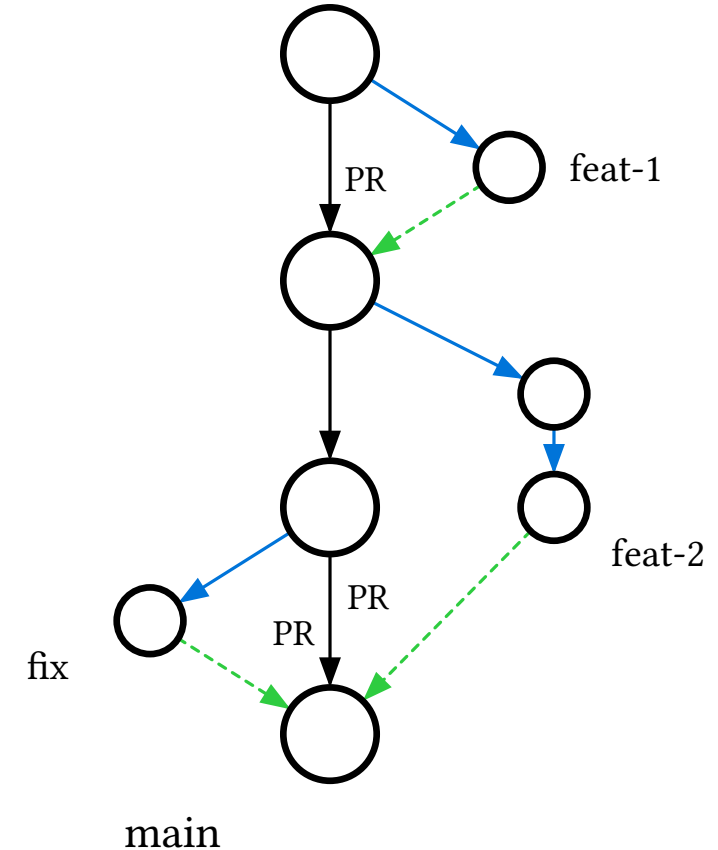
Centralized Workflow

- **Concept:** A single central repository serves as the “source of truth”.
- **Process:** All team members commit directly to the main branch.
- **Used** in small teams or solo projects where simplicity is key.
- **Risk** of conflicts and instability in the main branch



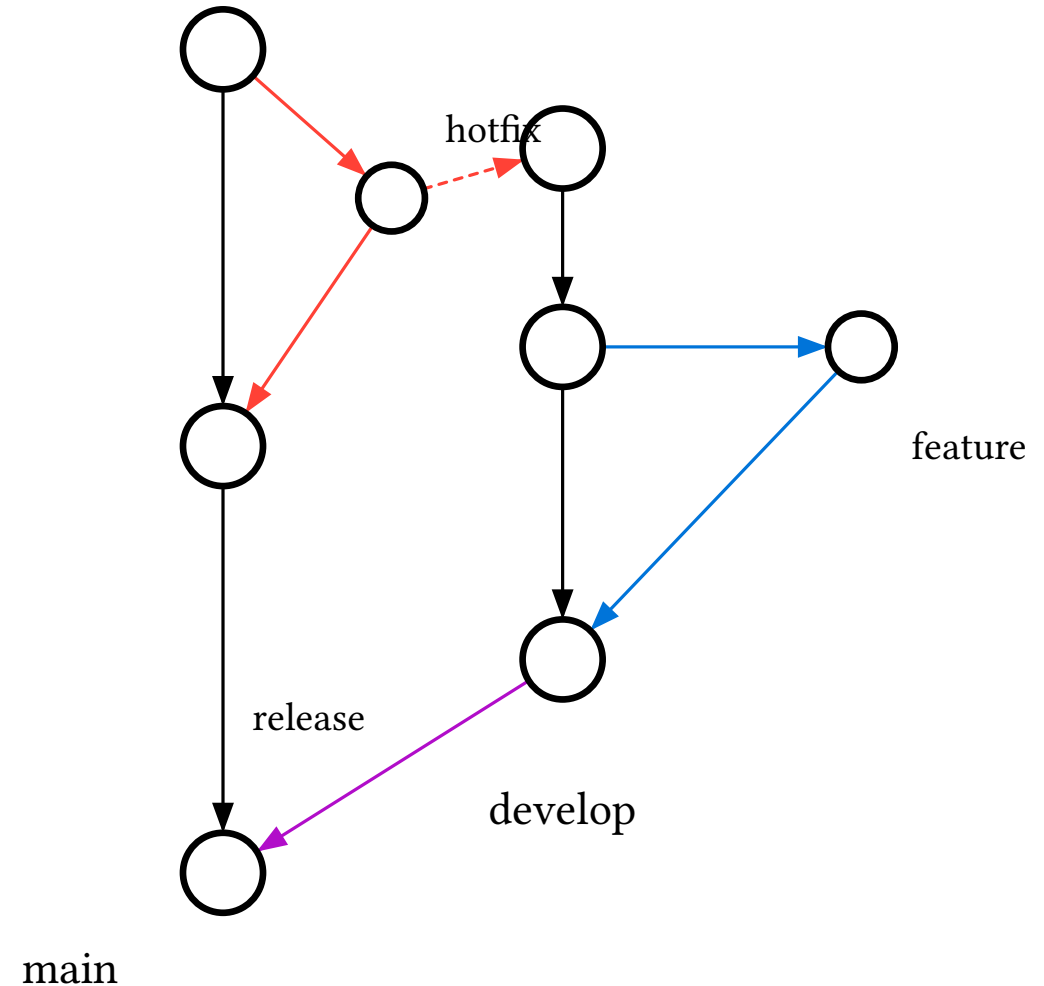
Feature Branch Workflow

- **Concept:** New work (features/fixes) happens in dedicated branches.
- **Process:** Changes are merged into main only after a code review.
- **Key Benefit:** The main branch is always kept in a stable, deployable state.
- **Risk:** Can lead to long-lived branches and integration issues if not regularly updated.



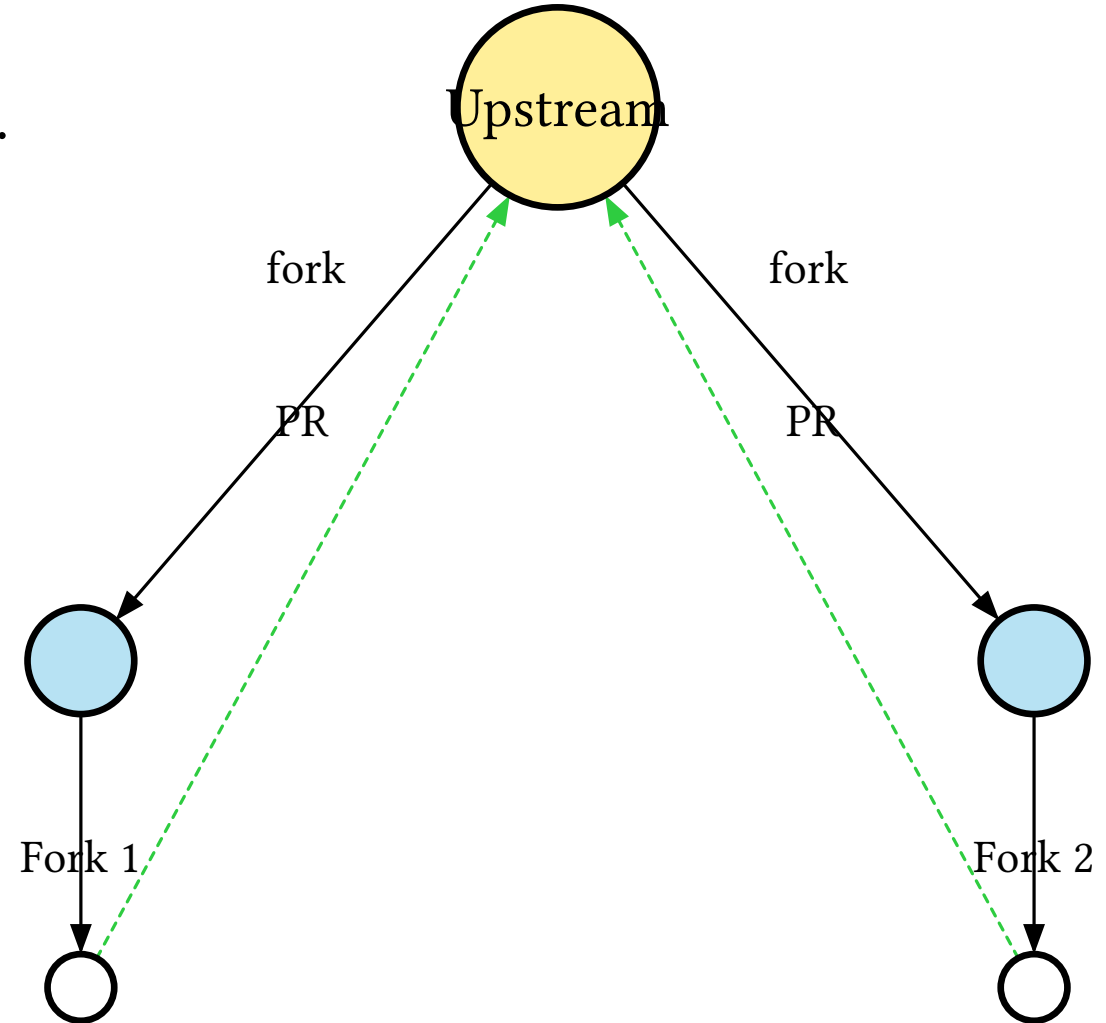
Gitflow Workflow

- **Concept:** A more structured workflow with specific branch types for different purposes.
- **Process:** A “strict” model using main, develop, feature, release, and hotfix branches.
- **Designed for** projects with a scheduled release cycle.
- **High control** but can be overkill for small, fast-moving teams.



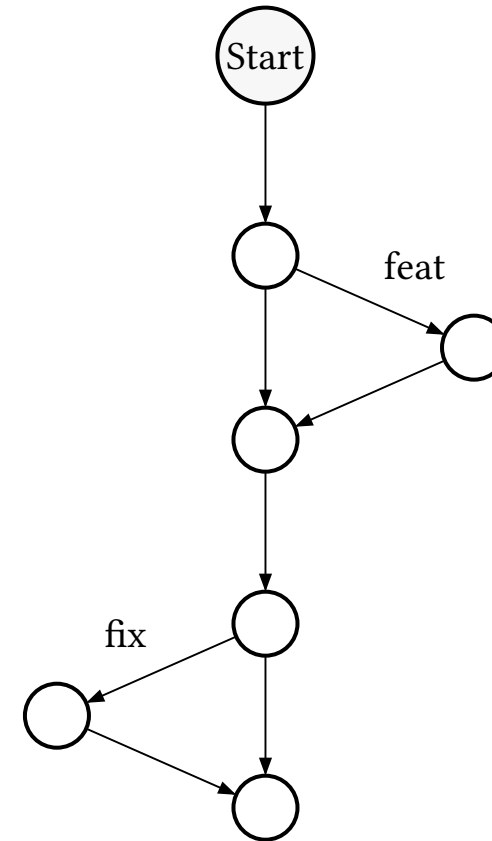
Forking Workflow

- **Concept:** Each developer has their own copy of the repository.
- **Collaboration:** Make changes to your fork, then open a PR to the original “upstream” repo.
- **Great for** open source projects, allows contributions without needing direct write access.
- **Maintainers** review and merge PRs, giving them control over the codebase.



Trunk-Based Development

- **Concept:** All developers work on a single shared branch (“trunk”).
- **Short-lived** branches are used to add features or fixes.
- **Continuous Integration** of changes through frequent merges.
- **Code review** and **testing** are critical to keep the trunk stable.



Questions to consider:

- How often will you integrate changes?
- What's your team's Git experience level?
- How will you handle code reviews and testing?

Workflow options:

- Centralized
- Feature Branch
- Gitflow
- Forking
- Trunk-Based Development
- Or a hybrid approach?

There's no single “right” answer! Choose what fits your experience and project requirements. You can always adapt as you learn!

Recap

- Git stores **snapshots** with parent pointers (0, 1, or 2+)
- **Branches** provide isolation for parallel development
- **HEAD** marks the current commit; **detached HEAD** lets you view past states
- Jump between commits with `git checkout <hash>`
- **Merge** combines branches, **Rebase** moves branch base
- Resolve conflicts by editing **conflict markers**
- Use **Git workflows** to organize team collaboration
- Consider your team's workflow strategy for your project!

Next week: More advanced Git topics and collaborative development!